

Code Explanation

Code Explanation: AWS Glue PySpark Job Test Suite This is a comprehensive test suite for an AWS Glue PySpark ETL job that processes customer and order data. Below is a clear breakdown of its structure and functionality.---

****Overview****

The test suite validates:- Data reading from S3- Data cleaning and deduplication- Schema validation and type enforcement- SCD Type 2 (Slowly Changing Dimension) operations- Business logic aggregations- Data Catalog registration- End-to-end ETL workflows---

****Test Structure****

****1. Test Fixtures (Setup)****

****Spark Session Fixture****

```
@pytest.fixture(scope="session")def spark():
```

- Creates a local Spark session for testing- Configured with 2 cores and 2 shuffle partitions for efficiency- Injects the session into Python's builtins so the job module can access it- Session persists across all tests (scope="session")##### ****Sample Data Fixtures****
Clean Customer Data:- 5 customers with complete information (custid, name, emailid, region)- Used to test normal processing flows
Customer Data with Nulls:- Contains NULL values, "Null" strings, and duplicate records- Tests data cleaning capabilities
Clean Order Data:- 5 orders with product details, prices, quantities, dates- Links to customers via custid
Order Data with Nulls:- Contains missing prices, null order IDs, invalid dates, duplicates- Tests order data cleaning---

****2. Test Classes****

****TestConfigManager****

Tests configuration loading functionality.****Key Tests:****- ``test_load_config_returns_dict``: Verifies config returns a dictionary with 'inputs' and 'outputs'- ``test_get_job_parameters``: Validates job parameters contain customer and order input configurations---

****TestS3DataReader****

Tests reading data from S3.****Key Tests:****
``test_read_data_safe_csv``: Mocks the Spark read chain: ``spark.read.format().option().load()``- Verifies CSV data is read correctly- Checks row count and column presence
``test_read_data_safe_normalizes_columns``: Creates DataFrame with uppercase column names (CustId, Name, etc.)- Verifies all columns are converted to lowercase- Ensures consistent column naming across datasets---

****TestS3DataWriter****

Tests writing data to S3.****Key Tests:****
``test_write_parquet``: Mocks the internal ``_write_parquet`` method- Verifies Parquet format writing is called correctly
``test_write_csv``: Mocks the internal ``_write_csv`` method- Tests CSV writing with options (e.g.,

header=true)**`test\_write\_empty\_dataframe`:** Verifies that attempting to write an empty DataFrame returns False- Prevents writing invalid/empty outputs---

****TestDataCleaner****

Tests data quality and cleaning operations.**Key Tests:**`test\_clean\_nulls\_removes\_null\_values`:** Verifies removal of NULL values and "Null"/"NULL" strings- Checks that cleaned data has fewer rows- Ensures no null values remain in the output**`test\_remove\_duplicates`:** Confirms duplicate rows are removed- Validates row count decreases after deduplication**`test\_clean\_dataframe\_full\_pipeline`:** Tests the complete cleaning workflow: 1. Remove NULL values 2. Remove "Null" strings 3. Remove duplicates- Verifies configuration-driven cleaning---

****TestSchemaValidator****

Tests schema definition and enforcement.**Key Tests:**`test\_get\_customer\_schema`:** Validates customer schema has 4 fields: custid, name, emailid, region- Confirms schema is a StructType**`test\_get\_order\_schema`:** Validates order schema has 6 fields including orderid, priceperunit- Ensures proper data types (DecimalType for prices)**`test\_apply\_schema`:** Tests applying schema to a DataFrame- Verifies row count remains unchanged after schema application---

****TestSCD2Manager****

Tests Slowly Changing Dimension Type 2 operations.**Key Tests:**`test\_add\_scd2\_columns`:** Verifies addition of SCD2 tracking columns: - `isactive`: Boolean flag for current records - `startdate`: Record effective start date - `enddate`: Record effective end date (NULL for active) - `opts`: Operation timestamp- Confirms all new records are marked as active (isactive=True)- Validates enddate is NULL for active records**`test\_write\_hudi\_table\_disabled`:** Tests behavior when Hudi (lakehouse format) is disabled- Returns False when Hudi writing is not enabled---

****TestAggregationEngine****

Tests business logic aggregations.**Key Tests:**`test\_calculate\_customer\_aggregate\_spend`:** Joins order and customer data- Calculates total spend per customer per date- Formula: `totalspend = SUM(priceperunit \* qty)` grouped by customer and date- Verifies output contains: name, date, totalspend columns**`test\_aggregate\_calculation\_accuracy`:** Uses specific test data to verify calculation accuracy- Example calculation: - Order 1: \$100.00 × 2 = \$200.00 - Order 2: \$50.00 × 3 = \$150.00 - **Total: \$350.00** - Confirms decimal precision is maintained---

****TestCatalogManager****

Tests AWS Glue Data Catalog integration.**Key Tests:**`test\_register\_table`:** Mocks Spark SQL execution- Verifies table registration in Glue Data Catalog- Tests parameters: database name, table name, S3 location---

****TestIntegration****

End-to-end integration tests.**Key Tests:**`test\_customer\_data\_pipeline`:** Complete customer processing workflow:1. **Clean**: Remove nulls, null strings, duplicates2. **Validate**: Apply customer schema3. **Transform**: Add SCD2 tracking columns4. **Verify**: Check final output has required columns**`test\_order\_data\_pipeline`:** Complete order processing workflow:1. **Clean**: Remove nulls and duplicates2. **Validate**: Apply order schema with proper types3. **Verify**: Confirm orderid and priceperunit columns exist**`test\_end\_to\_end\_aggregation`:** Full aggregation workflow:1. **Clean**:

Both customer and order datasets2. ****Join****: Combine datasets on custid3. ****Aggregate****: Calculate total spend per customer4. ****Verify****: All totals are positive numbers---

****TestSparkContextManager****

Tests Spark initialization.****Key Tests:****`test_initialize_spark_contexts``- Verifies initialization of: - SparkSession - GlueContext (AWS Glue-specific context) - Job object - Logger- Ensures all components are not None---

****Key Testing Patterns****

****1. Mocking****

Uses ``unittest.mock`` to simulate:- S3 read/write operations- Spark SQL execution- External dependencies

****2. Fixtures****

Provides reusable test data:- Clean datasets for happy path testing- Dirty datasets for data quality testing

****3. Assertions****

Validates:- Row counts (data cleaning effectiveness)- Column presence (schema compliance)- Data types (type enforcement)- Business logic accuracy (calculation correctness)

****4. Integration Testing****

Tests complete workflows to ensure:- Components work together correctly- Data flows through the entire pipeline- Final outputs meet requirements---

****Test Execution****

Run tests with:

```
pytest test_job.py -v
```

The ``-v`` flag provides verbose output showing each test result.